Bonus Lab 6: Advanced Virtual Memory Features

Instructor: Prof. Onur Mutlu, Dr. Mohammad Sadrosadati
TAs: Dr. Konstantina Koliogeorgi, Dr. Zhiheng Yue, Rahul Bera, Konstantinos Kanellopoulos, Nika Mansouri Ghiasi, Ataberk Olgun, Nisa Bostanci, Rakesh Nadig, Ismail Emir Yuksel, Haocong Luo, Mayank Kabra, Andreas Kosmas Kakolyris, Harshita Gupta, Konstantinos Marios Sgouras, Harsh Songara, Susana Rebolledo Ruiz, Spiros Galanopoulos

Corresponding TAs: Konstantinos Kanellopoulos, Spiros Galanopoulos
Assigned: Tuesday, December 9, 2025
Due: **Friday, January 9, 2026** (Midnight)

1. Introduction

In this lab, we will explore different features of the virtual memory (VM) subsystem to better understand its functionality both at the system software and hardware level. We will be using Virtuoso [1] (integrated with Sniper [2]), an architectural simulator that allows us to emulate the functionality of the OS and simulate the hardware components that support the VM subsystem. You will be working on the following techniques that span across the entire VM subsystem:

- (*Hardware-level*) **Replacement Policy for Page Table Entries**: The page table is a data structure used by the OS to map virtual addresses to physical addresses. Each process has its own page table, which is stored in physical memory. The CPU uses the page table to translate virtual addresses to physical addresses during memory accesses. Page table data is cached in the TLBs to reduce the latency of address translation. However, TLBs have limited capacity and need to evict entries when they are full. In this task, you will design and implement a replacement policy for the L2 cache that prioritizes PTEs over regular data cache lines insicde the L2 cache when runtime conditions demand it.
- (*Cross-layer*) **Page Table Placement Policy**: The OS is responsible for managing the page table and deciding where to place it in physical memory. The placement of the page table can have a significant impact on the performance of the VM subsystem. In this task, you will first (i) analyze the impact of conventional page table placement policies on system performance, and then (ii) design and implement a new page table placement policy that improves fairness across multiple applications running concurrently.

2. Lab Resources

2.1. How To Get Started

You need to use an Ubuntu-based machine to run the simulator. *(The simulator is not currently compatible latest MacOSes that make use of the ARM ISA.)* In case you want to run the simulator on a MacOS machine, you can use a virtual machine (VM) with Ubuntu installed. We suggest that you create an Ubuntu 20.04 LTS instance or later version using `https://www.qemu.org/download/#macos`.

You can use the following command to install the required dependencies on your machine:

Clone the Virtuoso (integrated with Sniper [2]) repository.

```
$ git clone https://gitlab.ethz.ch/kanellok/virtuoso_foca
$ git checkout comp_arch_lab6_2025
```

You need to install the dependencies for Virtuoso.

```
$ sh install_dependencies.sh
```

You need to run `build_virtuoso.sh`, which is a shell script to clean and build the simulator. Compile Virtuoso and Sniper using the following command. After you build the first time, you can use the `incremental_compile.sh` script to compile only the changed files in the future. In case you want to recompile the simulator from scratch, use `build_virtuoso.sh` instead of `incremental_compile.sh`.

```
$ ./build_virtuoso.sh
$ ./incremental_compile.sh
```

Let's test now if Virtuoso has been compiled successfully. You can run the following command to test the simulator.

```
$ sh ./run_example.sh
```

Now let's dive deeper into how to invoke the simulator and how to run an experiment:

```
$ ./run-sniper -c ./configuration_file.cfg -d ./example_output --genstats
  -s stop-by-icount:1000000 -- <workload>
```

Let's break down the command:

- `-c ./configuration_file.cfg`: This specifies the configuration file to use for the simulation. The configuration file contains various parameters that control the behavior of the simulator, such as the number of cores, the memory hierarchy, and the TLB configuration.
- `-d ./example_output`: This specifies the output directory where the simulation results will be stored. The simulator will create this directory if it does not already exist.
- `--genstats`: This option enables the generation of statistics at the end of the simulation. The file with the statistics (sim.stats) will be stored in the output directory specified by the `-d` option.
- `-s stop-by-icount:1000000`: This option specifies that the simulation should stop after 1000000 instructions have been executed. This is useful for debugging and testing purposes, as it allows you to run a short simulation or test a specific feature without running the entire workload.
- `-- <workload>`: This specifies the workload to run during the simulation. The workload can be any program or benchmark that you want to simulate.

2.2. Simulated Baseline Configuration

For the simulated system, we will use the following configuration which is stored in `./config/virtuoso_configs/virtuoso_reserve_thp.cfg` (Note: the configuration file includes other configuration files):

- Memory size: 48GB;
- Kernel space size: 32GB; // Kernel space takes 32GB of the total 48GB memory, leaving 16GB for user space;
- L1 TLB: 64 entries, 4-way;
- L2 TLB: 512 entries, 8-way;
- Memory Fragmentation: 0.0% (i.e., full fragmentation) up to 100% (i.e., no fragmentation) - setting this to 0.0% will not allow any 2MB pages to be allocated, while setting it to 100% will allow all 2MB pages to be allocated;
- Make sure that you simulate at least 20M instructions during each experiment.
- We suggest that you automate (i) bulk experiment launching and (ii) result analysis using a bash or python script.

You can find a detailed walkthrough for each component of interest at the following documentation web-

site: `https://cmu-safari.github.io/Virtuoso/`. You can also find the manual of Sniper at: `https://snipersim.org/documents/sniper-manual.pdf`.

You will be mainly working with three different subsystems:

- **`TLB Subsystem`**: This is the basic TLB subsystem that implements the TLB hierarchy, including the L1 and L2 TLBs.
- **`Memory Management Unit`**: This is the MMU subsystem that encompasses the memory management unit, including the page table walker, the page walk caches and the TLB subsystem.
- **`Physical Memory Allocator`**: This is the physical memory allocator subsystem that implements a reservation-based allocator for huge pages similar to [3].

3. Task 1: Page-Size Aware Replacement Policies (45 points)

3.1. Objective

Remember from our lecture on virtual memory that the TLB (Translation Lookaside Buffer) is a cache that stores recent translations of virtual addresses to physical addresses. Modern systems are using more than a single page size to improve performance and reduce TLB miss rates. For example, late kernel versions of Linux support both 4KB and 2MB pages, where the latter is often used for large memory allocations to reduce TLB pressure.

The utility of a TLB entry depends heavily on the page size it covers. A single entry for a 2MB page covers the same amount of memory address space as 512 entries for 4KB pages. Consequently, evicting a 2MB entry from the TLB hurts the *TLB Memory Reach* significantly more than evicting a 4KB entry. Standard replacement policies like LRU (Least Recently Used) treat all entries equally, strictly based on recency of access, ignoring the ”weight” or coverage of the translation.

Your goal in this task is to (i) understand the impact of different page sizes on performance, (ii) design a TLB replacement policy that recognizes the value of large pages, and (iii) co-design the Last Level Cache (LLC) replacement policy to minimize the penalty of TLB misses.

3.2. Task 1.1: Analyzing the Impact of Page Sizes on Address Translation Performance

In this task, you will analyze how different page sizes (e.g., 4KB vs. 2MB) impact L2 TLB performance (hit rates and memory reach) when mapping to physical frames of corresponding sizes.

You need to follow these steps to complete this task:

1. Design a microbenchmark that stresses the VM subsystem by accessing a large number of virtual addresses. You have freedom to design the microbenchmark *(Hint: Draw inspiration from Sparse Matrix Vector Multiplication or benchmarks that involve lots of pointer chasing)* but make sure that the L2 TLB misses per kilo instructions (MPKI) is larger than 10. This will ensure that the microbenchmark is stressing the L2 TLB subsystem. We recommend that you simulate at least 20M instructions in the microbenchmark. You can use the provided template in `microbenchmarks/` to get started. In the template, you can find instructions on how to record a trace of the microbenchmark execution that you can use to run the simulation. **Important Note:** *Make sure that the microbenchmark uses more than 8GB of memory to ensure that the L2 TLB is stressed.*
2. Run the microbenchmark on a simulated system using Virtuoso with a 4KB page size and collect the L2 TLB hit rate, the number of total execution cycles, and the amount of cycles spent on page table walks (i.e., the cycles spent on TLB misses).
3. Now we will enable both 4KB and 2MB pages in the system. Run the microbenchmark again on the same simulated system, but this time with both 4KB and 2MB pages enabled. Collect the same metrics as before plus a timeseries of the L2 TLB memory reach (i.e., the amount of memory captured by the L2 TLB over time across 10000 instruction epochs).

4. Analyze the results to understand how different page sizes affect system performance.

If you are not able to create a microbenchmark, you can use throughout the lab the following trace `traces/randacc_8gb.sift` directory.

In the report, include the following:

- A description of the microbenchmark and how it stresses the VM subsystem.
- Plots comparing the L2 TLB hit rates, total execution cycles, and cycles spent on page table walks between the two runs (4KB only vs. 4KB + 2MB).
- A plot of the L2 TLB memory reach timeseries and how it changes with the introduction of 2MB pages across intervals of 100000 instructions.

3.3. Task 1.2: Designing a Page-Size Aware TLB Replacement Policy

Standard replacement policies (like LRU or SRRIP) may evict valuable 2MB page entries to make room for 4KB entries, significantly reducing the TLB's effective memory reach. In this task, you must design a replacement policy that prioritizes retaining entries with large page sizes.

Requirements:

- Modify the L2 TLB logic to implement a replacement policy that favors 2MB pages over 4KB pages.
- For this task, tune the fragmentation of the physical memory allocator to make sure you have a "balanced" mix of 4KB and 2MB pages allocated (e.g., around 50% of each).
- You may design your policy **on top of the existing LRU/SRRIP or any policy that is already implemented in the simulator** (e.g., by making 2MB pages "sticky" or aging them slower) or you may design a new policy **from scratch**.
- For example: a policy might ensure that a 2MB page is only evicted if strictly necessary (e.g., if the set is full of only 2MB pages, or if the 2MB page has not been used for a very long time).

3.4. Task 1.3: Co-designing the LLC Replacement Policy

Optimizing the L2 TLB hit rate is crucial, but misses are inevitable. When an L2 TLB miss occurs, the hardware page table walker must fetch the Page Table Entries (PTEs) from memory (or from the page walk caches if available). If these PTEs are found in the Last Level Cache (LLC), the penalty of the page table walk is significantly reduced.

You must design a mechanism in the LLC to prioritize the retention of PTEs required for translations that are **currently missing** in the TLB.

Requirements:

- Modify the LLC replacement policy to distinguish between standard data requests and requests made by the hardware page table walker.
- The policy should prioritize caching PTEs (cache accessed by the page walker) over standard data, as these represent translations that do not exist in the TLB.
- You must ensure that this prioritization does not starve standard data accesses entirely, as this would degrade overall system performance.
- The hardest part of this task is to figure out if PTEs that exist in the LLC correspond to pages that are currently missing in the TLB. *Hint: You can track whether a translation resides in the TLB by maintaining a small structure (e.g., a Bloom filter or a probabilistic hardware structure) that records the physical/virtual addresses of pages get evicted from the TLB. When a page table walk request arrives at the LLC, you can check this structure to see if the corresponding page is currently missing in the TLB.*

3.5. Task 1.4: Implementation and Evaluation

1. **TLB Implementation:** Extend the L2 TLB to include your page-size aware replacement policy. You can find the L2 TLB implementation in the `simulator/sniper/common/core/memory_subsystem/parametric_dram_directory_msi/translation_components/` directory.
2. **LLC Implementation:** Extend the LLC to implement the PTE-prioritizing replacement policy. You can identify page walker requests by checking the request type.
3. **Evaluation:** Run the microbenchmark from Task 1.1 (or the provided trace) using your new combined TLB and LLC policies.

In the report, include the following:

- A detailed description of your TLB replacement algorithm. How do you decide which entry to evict? How do you prevent starvation of 4KB pages?
- A description of your LLC co-design and replacement policy. How does the LLC identify which PTEs to prioritize? How do you balance between PTEs and standard data?
- A quantitative comparison: Compare the **TLB Hit Rate** and **Total Execution Cycles** of your new design against the baseline (Default replacement policy in TLB and LLC).

Files of interest (You can search for the string "CompArch" or "Lab 6" to hover around relevant code parts):

`cache_set_tlb_template.cc`: TLB set implementation (Modify replacement logic here)
`cache_set_llc_template.cc`: LLC set implementation (Modify replacement logic here)
`cache_set_srrip.cc`: SRRIP replacement policy implementation (You can build your TLB/LLC policy on top of this if you want) [4]
`tlb.cc`: TLB implementation (Modify replacement logic here)
`virtuoso_pagetable_aware.cfg`: Configuration of the page table aware placement policy in DRAM
`virtuoso_tlb_replacement.cfg`: Configuration of the TLB replacement policy (change it accordingly if needed)
`virtuoso_llc_replacement.cfg`: You should create a new configuration file for your LLC replacement policy and set it here.

4. Task 2: DRAM-Aware Page Table Placement (40 points)

4.1. Objective

Optimizing the replacement policy (Task 1) reduces the number of page walks, but it cannot eliminate them entirely. When a hard TLB miss occurs, the hardware walker must issue memory requests to read the Page Table Entries (PTEs) from DRAM.

The DRAM Row Buffer Problem: Recall that DRAM is organized into Ranks, Banks, Rows, and Columns. To access data, a Row must be "activated" (opened) into the Row Buffer.

- **Row Buffer Hit:** Accessing data in an already open row is fast (≈15-30ns).
- **Row Buffer Conflict/Miss:** Accessing data in a closed row (or a different row in the same bank) requires precharging the bank and activating the new row, which is slow (≈40-60ns).

In standard systems, the Operating System allocates physical frames for Page Tables and Application Data indiscriminately from the same free list. This leads to a scenario where a Page Table entry might reside in *DRAM Bank 0, Row 100*, while the application data it points to resides in *DRAM Bank 0, Row 200*. During a page table walk, the memory controller might have to close Row 200 (Data), open Row 100 (PTE), and then immediately close Row 100 to reopen Row 200 to service the actual instruction. This "thrashing" of the row buffer significantly increases the effective memory latency.

Your goal in this task is to design a **Page Table Isolation Policy**. You must modify the physical frame allocator to place Page Tables in dedicated DRAM regions (e.g., specific Banks or Rows) to ensure they do not compete for Row Buffers with regular application data.

4.2. Task 2.1: Analyzing the Address Mapping Scheme

Before implementing the placement policy, you must understand how physical addresses map to DRAM coordinates (Channel, Rank, Bank, Row, Column) in the simulator.

1. Inspect the memory controller configuration and the address mapping function in the simulator code. Determine which bits of the physical address are used as the **Bank Selector** and which are used as the **Row Selector**.
2. Calculate the physical address ranges that map to specific DRAM Banks. For example, if the system has 8 Banks, determine which physical addresses land in Bank 7.

4.3. Task 2.2: Implementing DRAM-Aware Page Table Allocation

You need to modify the Operating System's physical frame allocator (simulated in Virtuoso) to enforce isolation.

Requirements:

- **Reserved Region:** Designate a specific portion of DRAM (e.g., the last bank of every rank) exclusively for page table pages.
- **Split Allocation Logic:**
  - When the OS needs to allocate a page to store *Page Table Entries* (for the page walker), it must allocate a physical page from the **Reserved Region**.
  - When the OS needs to allocate a page for *Application Data* (stack, heap, code), it must allocate a physical page from the **Standard Region**.
- **Goal:** Ensure that a stream of data accesses and a stream of page table walk accesses generally target different Banks, allowing them to proceed in parallel (Bank-Level Parallelism) without causing row buffer conflicts.

Hint: The simplest implementation is "Bank Isolation". If your DRAM has 8 Banks (indices 0-7), configure the allocator to place all Page Tables in Bank 7, and all Data in Banks 0-6. To achieve this, you need to manipulate the physical address bits corresponding to the bank selector during allocation. An easy way is to create a list of free pages that belong to Bank 7 and whenever a page table page is requested, allocate from that list.

4.4. Task 2.3: Evaluation

1. **Implementation:** Modify the memory allocator logic. In the simulation framework, look for the function responsible for allocating physical pages (often named `allocate_physical_page` or similar in the MMU/OS emulation files).
2. **Testing:** Run the same microbenchmark from Task 1.1. This benchmark has high TLB pressure, meaning it generates frequent page walks intermixed with data accesses. For this experiment, tune the fragmentation of the physical memory allocator to make sure the system is using *only 4KB pages* (i.e., set fragmentation to 0.0).
3. **Metrics:** Collect the **DRAM Row Buffer Hit Rate** specifically for:
   - Requests marked as `PAGE_TABLE` (Page Table Walks).
   - Requests marked as `NON_PAGE_TABLE` (Standard Data).
   - You can find the request types in `cache_block_info.h`.

In the report, include the following:

- A diagram or explanation of the Physical Address → DRAM Mapping scheme used in your simulation (which bits correspond to Bank/Row).
- Code snippets showing how you modified the allocator to separate Page Tables from Data.
- A quantitative comparison:
  1. **Baseline:** Row Buffer Hit Rate for Page Walks vs. Data (mixed allocation).
  2. **Your Design:** Row Buffer Hit Rate for Page Walks vs. Data (isolated allocation).
- Did the overall application performance (IPC) improve? Why or why not? (Consider if creating a "hot bank" for page tables introduced a new bottleneck).

Files of interest:

`dram_perf_model_detailed.cc.cc`: DRAM logic (Check address mapping here)
`ddr4_2400.cfg`: Configuration of DRAM parameters (Number of Banks, Ranks, etc.)
`page_table_aware.cfg`: Configuration of the page table-aware placement policy in DRAM
`pagetable_aware_allocator.cc`: Physical frame allocator template (to be modified for this task)
`page_fault_handler.cc`: Page fault handling logic (you may need to modify this depending on your allocator implementation)

5. Tips

- If needed, please ask questions to the TAs via the online Q&A forum in Moodle (`https://moodle-app2.let.ethz.ch/mod/moodleoverflow/view.php?id=1315279`).
- When you encounter a technical problem, please first read the error messages. A search on the web can usually solve many debugging issues, and error messages.

5.1. Reading statistics

You can read the statistics generated by the simulator using the `sim.stats` file. This file is generated at the end of the simulation and contains all the statistics collected during the simulation.

These are some of the most important statistics that you should look at:

- `mmu.total_table_walk_latency` - The total latency of all the page table walks that were performed during the simulation.
- `TLB_L2_1.miss` - The number of L2 TLB misses that occurred (i.e., the number of page table walks that were performed).
- `TLB_L2_1.hit` - The number of L2 TLB hits that occurred.
- `mmu.total_translation_latency` - The total latency of all the translations that were performed during the simulation.
- `performance_model.cycle_count` - The total number of cycles that the simulation took to run.
- `ddr.page-conflict-data-to-data` - The number of page conflicts in DRAM between data requests.
- `ddr.page-conflict-data-to-metadata` - The number of page conflicts in DRAM between data and metadata requests.
- `ddr.page-conflict-metadata-to-data` - The number of page conflicts in DRAM between metadata and data requests.
- `ddr.page-conflict-metadata-to-metadata` - The number of page conflicts in DRAM between metadata requests (i.e., page table entries).
- `ddr.page-hits` - The number of page hits in DRAM (i.e., when a request accesses a row that is currently open).

- `ddr.page-miss` - The number of page miss events in DRAM (i.e., when a request accesses a row that is not currently open and causes a row conflict).
- `ddr.page-empty` - The number of page empty events in DRAM (i.e., when a request accesses a row that is not currently open).
- `L1-D.loads-where-page-table-dram-local` - The number of loads that led to fetching page table entries from local DRAM.
- `L1-D.loads-where-page-table-L2` - The number of loads that led to fetching page table entries from L2 cache.
- `L1-D.loads-where-page-table-nuca-cache` - The number of loads that led to fetching page table entries from NUCA cache.
- `L1-D.loads-where-dram-local` - The number of loads that led to fetching data from local DRAM.
- `L1-D.loads-where-L1` - The number of loads that led to fetching data from L1 cache.
- `L1-D.loads-where-L2` - The number of loads that led to fetching data from L2 cache.
- `L1-D.loads-where-nuca-cache` - The number of loads that led to fetching data from NUCA cache.

5.2. Extending the simulator with new statistics

First, you need to define a variable in the file where the component of interest is defined to hold the statistic you want to collect. For example, if you want to collect a statistic for the L2 TLB, you would define a variable in the constructor of `tlb.h` file like this:

```
$ UInt64 variable;
```

Then, you need to register the variable as a statistics metric in the component of interest. Be careful to use the correct component name and core ID. The component name is the name of the component that you want to register the statistic for, and the core ID is the ID of the core that you want to register the statistic for. For example, if you want to register a statistic for the L2 TLB of core 0, you would use the following command:

```
$ registerStatsMetric("l2tlb", 0, "variable", variable);
```

The general format for registering a statistic is as follows:

```
$ registerStatsMetric("component_name", core_id, "metric_described_by_variable", variable);
```

5.3. Extending the simulator with new configurations

When you need to add a new configuration to the simulator you can do so in two main ways:

- You can add a new configuration to the `simulator/sniper/config/virtuoso_configs/` directory. For example, call it `my_config.cfg` and copy the contents of `simulator/sniper/config/virtuoso_configs/virtuoso_reserve_thp.cfg` to it.
  Then, you can modify the configuration file to include your new configuration parameters like this:

  ```
  my_config.cfg:
  [perf_model/my_component]
  number_of_entries = 64
  ```

- or when you execute Sniper, you can pass the new configuration parameters as command line arguments:

  ```
  $ ./run-sniper -c ./config/virtuoso_configs/virtuoso_reservethp.cfg
    -g --perf_model/my_component/number_of_entries=64 -- my_program
  ```

5.4. Extracting simulation results

Every time you run a new simulation, you should see at least five new files:

- `sim.cfg` - This contains the CPU parameters used in the simulation.
- `sim.info` - This contains information about the simulator and command line parameters that were used to run the simulation.
- `sim.out` - This contains statistics about the program and the performance of various components of the processor.
- `sim.stats.sqlite3` - This is a database file that contains all the data and metrics gathered during simulation. It is read and processed by the various analysis tools included with Sniper.
- `sim.stats` - This is a text file that contains the same data as `sim.stats.sqlite3`, but in a human-readable format. It is useful for quick checks and debugging.
- `heartbeat.txt` - This file is created by the Sniper simulator to indicate that the simulation is still running. It is updated periodically during the simulation.

5.5. Recompiling from scratch

If everything fails, you can always run the following command to purge all the built files and re-compile from scratch (you can use the scripts `build_virtuoso.sh` and `incremental_compile.sh` to compile the simulator again):

```
$ make distclean
$ make clean
$ make -j
```

References

[1] Konstantinos Kanellopoulos, Konstantinos Sgouras, F. Nisa Bostanci, Andreas Kosmas Kakolyris, Berkin Kerim Konar, Rahul Bera, Mohammad Sadrosadati, Rakesh Kumar, Nandita Vijaykumar, and Onur Mutlu. Virtuoso: Enabling fast and accurate virtual memory research via an imitation-based operating system simulation methodology. In *ASPLOS*, 2025.

[2] Trevor E. Carlson, Wim Heirman, and Lieven Eeckhout. Sniper: Exploring the Level of Abstraction for Scalable and Accurate Parallel Multi-Core Simulations. In *SC*, 2011.

[3] Juan Navarro, Sitaram Iyer, Peter Druschel, and Alan Cox. Practical, Transparent Operating System Support for Superpages. In *OSDI*, 2002.

[4] Aamer Jaleel, Kevin B. Theobald, Simon C. Steely, and Joel Emer. High Performance Cache Replacement Using Re-Reference Interval Prediction (RRIP). 2010.